

Guia de apresentação da pipeline de segmentação

Este documento explica, em linguagem de apresentação, como funciona a solução do projeto **pdiseg**, quais etapas existem, por que cada técnica foi escolhida e como usar a execução por Docker. O objetivo é servir como roteiro para explicar a implementação ao professor.

1. Objetivo do projeto

O projeto segmenta rótulos em imagens industriais de embalagens de frango. A entrada é uma imagem em escala de cinza, normalmente com várias embalagens dentro de uma caixa. A saída é um ou mais recortes salvos em **result/<Classe>/**, cada recorte contendo um cluster de rótulo válido.

O alvo atual não é apenas o texto isolado do produto. O alvo é o **label cluster**:

- o rótulo do tipo do produto, por exemplo **PEITO**, **MOELA**, **ASAS**, etc.;
- e, quando estiver visualmente junto, o selo/marca **SUPER FRANGO** ao lado ou acima.

Uma detecção só é considerada válida quando contém evidência do rótulo do produto. Um crop contendo apenas **SUPER FRANGO**, tabela nutricional, borda da caixa, brilho do plástico ou texto aleatório é considerado falso positivo.

2. Restrições importantes

A solução segue a proposta da disciplina de PDI:

- não usa OCR;
- não usa rede neural;
- não usa modelo treinado;
- não usa o nome da pasta/classe para decidir o que detectar;
- usa técnicas clássicas: filtros, equalização, limiarização, morfologia, componentes conectados, histogramas e medidas geométricas.

Essa decisão é importante para defender a solução: o algoritmo não “lê” o produto. Ele procura padrões visuais compatíveis com um rótulo escuro com texto claro e contexto de embalagem.

3. Diferença entre pipeline e calibrate

Pipeline

A pipeline é a execução de produção. Ela percorre todas as imagens do dataset, detecta os rótulos e salva os crops finais.

Entrada:

`data/Train_and_Validation/<Classe>/<imagem>.jpg`

Saída:

`result/<Classe>/<imagem>_segmented_1.png`

Com Docker:

`make docker-up THREADS=1 DOCKER_CPUS=0.75 DOCKER_NICE=10`

Calibrate

O **calibrate** é uma execução de auditoria/depuração. Ele roda o mesmo detector, mas em vez de salvar só crops finais, salva materiais para análise:

- overlays com caixas desenhadas;
- `boxes.json` com candidatos, âncoras e caixas finais;
- `stats.csv` com estatísticas por classe.

Ele não é o resultado final da entrega. Ele existe para visualizar e justificar o que a pipeline está fazendo.

Com Docker:

```
make docker-calibrate THREADS=1 DOCKER_CPUS=0.75 DOCKER_NICE=10
```

4. Visão geral do fluxo

O fluxo por imagem é:

imagem original

- > carregamento
- > pré-processamento
- > geração de máscaras
- > componentes conectados
- > caixas candidatas
- > filtro geométrico
- > scoring
- > extração da âncora do produto
- > expansão para label cluster
- > seleção final
- > crop na imagem original

O ponto principal é que a pipeline detecta usando uma imagem de trabalho melhorada, mas o crop final é retirado da imagem original. Isso preserva o conteúdo real da imagem entregue, sem equalização artificial ou regiões mascaradas no resultado final.

5. Carregamento da imagem

Arquivo principal: `src/pdiseg/io/dataset.py`.

O carregamento faz:

- encontra imagens dentro das pastas de classe;
- ignora arquivos auxiliares como `Zone.Identifier`;
- lê a imagem com `imageio`;
- garante que a imagem usada pelo detector seja 2-D e `uint8`.

Por que isso foi usado:

- o dataset tem estrutura fixa por classe;
- as classes servem apenas para organizar saída;
- o detector precisa receber uma matriz de intensidade simples para aplicar PDI clássica.

6. Pré-processamento

Arquivo principal: `src/pdiseg/detection/preprocess.py`.

O pré-processamento gera três imagens:

Campo	Função
<code>gray</code>	imagem original em escala de cinza
<code>clahe</code>	imagem equalizada para realçar contraste
<code>work</code>	imagem usada para gerar máscaras e candidatos

6.1 Conversão para cinza

Se a imagem tiver mais de um canal, o código usa o primeiro canal. O dataset é essencialmente grayscale, então essa etapa padroniza a entrada.

Objetivo:

- garantir que todos os filtros recebam uma matriz 2-D;
- evitar diferenças de comportamento entre imagens grayscale e RGB.

6.2 Filtro de mediana

Técnica usada: `scipy.ndimage.median_filter`, tamanho 3.

O que faz:

- substitui cada pixel por uma mediana local;
- reduz ruídos pontuais, como pixels muito claros ou escuros isolados.

Por que foi usado:

- ruídos pequenos podem virar componentes conectados falsos;
- a mediana preserva bordas melhor que uma média simples;
- é uma técnica clássica de filtragem espacial.

6.3 CLAHE

Técnica usada: `skimage.exposure.equalize_adapthist`.

O que faz:

- aumenta o contraste local da imagem;
- melhora a separação visual entre texto, rótulo e fundo.

Por que foi usado:

- algumas embalagens têm iluminação irregular;
- rótulos e letras podem aparecer com baixo contraste;
- equalização adaptativa melhora regiões locais sem depender de uma única equalização global.

6.4 Máscara do contador FPS

As imagens têm um contador de FPS queimado no canto superior esquerdo. Essa região não é rótulo de produto.

O que a pipeline faz:

- substitui a região do FPS pela mediana da imagem de trabalho.

Por que foi usado:

- o texto do FPS tem alto contraste;
- sem essa remoção, ele poderia virar candidato falso;
- preencher com a mediana evita criar uma borda artificial muito escura ou muito clara.

6.5 Exclusão horizontal lateral

Arquivo auxiliar: `src/pdiseg/detection/roi.py`.

O código ignora faixas laterais da imagem:

- `exclude_left_frac = 0.20;`
- `exclude_right_frac = 0.08.`

O que faz:

- preenche colunas laterais com a mediana da região central;
- zera máscaras fora da região de interesse.

Por que foi usado:

- nas laterais aparecem caixa, papelão, bordas da esteira e textos cortados;
- esses elementos geravam falsos positivos;
- a câmera e o enquadramento são fixos, então uma região horizontal válida é aceitável como filtro espacial clássico.

7. Geração de máscaras candidatas

Arquivo principal: `src/pdiseg/detection/masks.py`.

Depois do pré-processamento, a pipeline cria várias máscaras binárias. Cada máscara procura um tipo de evidência visual.

7.1 Máscara de densidade de texto (`text_density`)

Técnica:

`pixel > média_local + offset`

O que faz:

- calcula uma média local com `uniform_filter`;
- marca pixels muito mais claros que a vizinhança.

Objetivo:

- encontrar letras claras em cima de regiões mais escuras;

- detectar texto do rótulo do produto.

Por que foi usada:

- os rótulos de produto costumam ter texto claro sobre fundo escuro;
- é simples, clássica e não depende de OCR.

7.2 Máscara de luminância escura (`dark_luma`)

Técnica:

`work <= percentil escuro`

O que faz:

- calcula um percentil global de intensidade;
- marca regiões mais escuras da imagem.

Objetivo:

- localizar corpos escuros que podem ser os badges/rótulos.

Por que foi usada:

- o rótulo do produto normalmente é uma placa escura;
- ajuda a diferenciar rótulo de plástico claro ou fundo da embalagem.

7.3 Black-hat morfológico (`black_hat`)

Técnica: `skimage.morphology.black_tophat`.

O que faz:

- destaca estruturas escuras pequenas em fundo mais claro.

Objetivo:

- encontrar detalhes escuros e bordas internas de rótulos;
- complementar a limiarização por luminância.

Por que foi usado:

- algumas embalagens têm sombras e texturas;
- o black-hat ajuda a capturar detalhes escuros que um limiar simples perderia.

7.4 Máscara de brilho (`glare`)

Técnica:

`pixel >= percentil alto`

O que faz:

- marca regiões muito claras, geralmente reflexos no plástico.

Objetivo:

- evitar que brilhos virem candidatos.

Por que foi usada:

- embalagem plástica gera reflexos fortes;
- reflexos têm alto contraste e poderiam parecer texto.

7.5 Máscara combinada (combined)

Combinação:

```
(text_density | dark_luma | black_hat) & ~glare
```

Depois aplica:

- fechamento morfológico;
- remoção de componentes encostados na borda (`clear_border`);
- restrição da região horizontal de interesse.

Objetivo:

- juntar evidências diferentes em uma máscara principal;
- agrupar letras e fundo do rótulo em regiões candidatas.

Por que foi usada:

- nenhuma máscara isolada é suficiente para todos os casos;
- combinar sinais aumenta recall;
- remover glare e bordas reduz falsos positivos.

7.6 Máscara de densidade de bordas (edge_density)

Técnica:

- filtro de mediana;
- gradiente Sobel;
- magnitude do gradiente;
- densidade local por janela;
- fechamento e abertura morfológica.

Objetivo:

- detectar regiões com muitas bordas, típicas de texto.

Por que foi usada:

- texto tem várias transições claro/escuro;
- ajuda quando a intensidade absoluta do rótulo varia;
- é uma característica clássica de textura/borda.

7.7 Máscara DoG-style de texto (dog_text)

Apesar do nome, a implementação atual usa subtração de fundo por janela larga:

```
background = uniform_filter(image, janela_grande)
text = (image - background) > threshold
```

Depois agrupa texto com abertura, dilatação e fechamento.

Objetivo:

- capturar letras claras sobre fundo localmente escuro.

Por que foi usada:

- alguns rótulos aparecem escuros em relação ao entorno, mas a iluminação global varia;
- comparar com o fundo local é mais robusto que usar apenas limiar global;
- continua sendo PDI clássica: filtro, subtração, threshold e morfologia.

8. Componentes conectados e caixas candidatas

Arquivo principal: `src/pdiseg/detection/candidates.py`.

Depois de criar as máscaras, o algoritmo usa componentes conectados:

máscara binária -> `label()` -> `find_objects()` -> bounding boxes

O que faz:

- cada região conectada vira uma caixa (`x, y, w, h`);
- regiões pequenas demais são descartadas;
- regiões grandes demais ou absurdas são filtradas;
- caixas muito sobrepostas são deduplicadas.

Objetivo:

- transformar pixels segmentados em hipóteses geométricas de rótulo.

Por que foi usado:

- componentes conectados são uma técnica clássica de segmentação;
- bounding boxes são simples de avaliar, pontuar e recortar.

9. Filtro geométrico

Função: `keep_label_clusters`.

Antes de pontuar, o algoritmo descarta caixas incompatíveis por geometria:

- área mínima;
- área máxima;
- elongação máxima;
- margem lateral, quando configurada.

Objetivo:

- remover candidatos claramente impossíveis, como linhas finas, bordas compridas ou regiões gigantes.

Por que foi usado:

- reduz custo do pós-processamento;
- reduz falsos positivos antes do scoring;
- usa apenas propriedades geométricas, sem semântica ou OCR.

10. Scoring dos candidatos

Arquivo principal: `src/pdiseg/detection/scoring.py`.

Cada caixa candidata recebe um score entre 0 e 1. O score combina várias características.

10.1 Densidade escura

Mede quanto da região é escura em relação ao percentil global.

Por que importa:

- o rótulo do produto costuma ter corpo escuro.

10.2 Densidade de texto

Mede pixels com contraste local positivo.

Por que importa:

- o rótulo precisa ter texto, não apenas uma mancha escura.

10.3 Densidade de bordas

Usa Sobel dentro da caixa.

Por que importa:

- letras e logotipos têm muitas bordas;
- regiões lisas ou sombras têm menos estrutura.

10.4 Textura

Usa desvio padrão da intensidade.

Por que importa:

- rótulos possuem variação interna;
- áreas homogêneas são menos prováveis.

10.5 Fração de brilho

Mede pixels muito claros/reflexivos.

Por que importa:

- brilho de plástico pode gerar bordas falsas;
- regiões com muito glare são penalizadas.

10.6 Contraste com entorno

Compara a média interna da caixa com uma região ao redor.

Por que importa:

- rótulos costumam contrastar com a embalagem.

10.7 Fundo aberto (`opened_background`)

Usa abertura morfológica para estimar o fundo local.

Por que importa:

- permite medir letras claras sobre fundo escuro;
- ajuda a validar texto claro real, não ruído isolado.

10.8 Bright-on-dark

Mede pixels mais claros que o fundo aberto por um offset.

Por que importa:

- é uma das evidências mais fortes de rótulo de produto.

10.9 Extent por Otsu

Aplica Otsu dentro da região e mede a fração de foreground escuro.

Por que importa:

- rótulos tendem a ter uma proporção equilibrada de fundo e texto;
- tabelas, sombras e regiões lisas têm proporções diferentes.

10.10 Bimodalidade

Também usa Otsu, mas mede:

- contraste entre classe clara e escura;
- equilíbrio entre as duas classes.

Por que importa:

- rótulo com letras claras em fundo escuro é naturalmente bimodal;
- isso ajuda principalmente em casos em que `bright_on_dark` é fraco.

11. Pós-processamento

Arquivo principal: `src/pdiseg/detection/postprocess.py`.

O pós-processamento é a parte que transforma candidatos pontuados em caixas finais.

11.1 Ideia principal

A saída final precisa ser o cluster local do rótulo, mas sem aceitar brand-only. Por isso a pipeline trabalha em duas ideias:

1. primeiro encontra uma **âncora de produto**;
2. depois expande essa âncora para o **label cluster**.

A âncora é a evidência obrigatória de produto. O cluster é a geometria final.

11.2 Extração da âncora de produto

Função: `extract_product_anchor`.

O que faz:

- recebe uma caixa candidata;
- internamente chama `refine_to_name_label`;
- procura uma região mais provável de ser o rótulo do produto;
- valida essa região por gates visuais.

Por que foi usado:

- candidatos iniciais podem conter marca, produto, plástico e texto ao mesmo tempo;
- validar o cluster inteiro poderia aceitar regiões grandes demais;
- validar a âncora garante que existe produto dentro do crop final.

11.3 `refine_to_name_label`

Essa função:

- aplica Otsu na região;
- procura componentes escuros;
- usa fechamento/abertura morfológica para formar placas;
- testa candidatos por preenchimento, área, aspecto e texto claro.

Objetivo:

- encontrar a parte interna que parece o rótulo de produto.

Por que foi mantida:

- mesmo que a saída final seja o cluster, a âncora ainda é necessária para impedir falsos positivos.

11.4 Gates estritos

A âncora passa por validações como:

- texto claro em fundo escuro;
- nível de fundo suficientemente escuro;
- bordas suficientes;
- extent mínimo;
- área e aspecto plausíveis.

Objetivo:

- alta precisão;
- impedir brand-only e ruídos.

11.5 Gate relaxado de recuperação

Se a busca estrita não acha nada, a pipeline tenta um gate relaxado, mas ainda ancorado.

Ele exige:

- área mínima maior;

- densidade de texto;
- densidade de borda;
- bimodalidade;
- limite de extent;
- não estar colado nas bordas do frame.

Por que existe:

- alguns rótulos reais têm baixo contraste;
- sem recuperação, alguns frames visíveis ficariam sem crop;
- ainda é mais seguro que escolher a maior mancha escura.

Importante: não existe fallback para maior componente escuro arbitrário. Se não há âncora de produto, a pipeline prefere não emitir crop.

11.6 Expansão para label cluster

Função: `expand_to_label_cluster`.

Depois que a âncora é validada, a pipeline procura contexto local:

- acima;
- na diagonal superior;
- lateralmente;
- contendo/tocando a âncora.

Ela só junta contexto se houver evidência de texto, borda ou bimodalidade.

Objetivo:

- incluir a marca **SUPER FRANGO** quando ela faz parte do mesmo rótulo local;
- evitar crop muito apertado só no produto;
- ainda impedir regiões distantes ou tabelas aleatórias.

Por que foi usada:

- o requisito atual aceita e prefere o cluster completo;
- o brand ajuda visualmente a entender que o crop é do rótulo correto;
- mas a validação continua ancorada no produto, não na marca.

11.7 NMS e seleção final

A pipeline usa Non-Maximum Suppression (NMS) para remover caixas sobrepostas.

Configuração atual:

- `primary_cluster_only=True`;
- normalmente emite um cluster principal por frame.

Objetivo:

- reduzir falsos positivos;
- evitar vários crops parciais do mesmo rótulo;
- priorizar qualidade da detecção em vez de quantidade.

12. Crop final

Mesmo que a detecção use imagem equalizada/mascarada, o crop é retirado da imagem original.

Por que isso é importante:

- a entrega deve representar o dado original;
- CLAHE e máscaras são apenas auxiliares de detecção;
- o resultado final não deve conter o FPS mascarado artificialmente nem contraste alterado.

13. Calibração e auditoria visual

Arquivo principal: `src/pdiseg/calibration/service.py`.

O `calibrate` roda `inspect_frame`, que retorna:

- `candidates`: caixas candidatas brutas;
- `kept`: âncoras de produto selecionadas;
- `labels`: clusters finais.

Nos overlays:

Cor	Significado
vermelho	candidatos rejeitados ou intermediários
amarelo	âncora de produto selecionada
verde	cluster final salvo como detecção

Arquivos gerados:

- `calibration/<Classe>/<imagem>_overlay.png`;
- `calibration/boxes.json`;
- `calibration/stats.csv`.

Para que serve

Serve para demonstrar e auditar o detector:

- ver onde ele encontrou candidatos;
- ver se a âncora está no produto;
- ver se o cluster final inclui contexto correto;
- contar quantas detecções houve por classe;
- identificar imagens sem label ou com falso positivo.

Por que não é a entrega final

O `calibrate` é ferramenta de revisão. A entrega final da segmentação é a pasta `result/`, gerada pela pipeline.

14. Review viewer

O review viewer (`pdiseg-review`) é uma interface web local para navegar:

- imagem original;

- overlay de calibração;
- crops finais.

Ele não roda o detector de novo. Ele só lê os artefatos já gerados.

Com Docker:

```
make docker-review
```

15. Como executar com Docker

O `.env` do projeto está preenchido com:

```
DATA=./data/Train_and_Validation
OUT=./result
CALIB=./calibration
LIMIT=9999
MAX_IMAGES=
OFFSET=0
PROGRESS_EVERY=25
THREADS=1
DOCKER_CPUS=0.75
DOCKER_MEMORY=2g
DOCKER_NICE=10
PORT=8765
DOCKER_UID=1000
DOCKER_GID=1000
```

Rodar a pipeline completa

```
make docker-up
```

Rodar a calibração completa

```
make docker-calibrate
```

Rodar sem travar o notebook

As variáveis importantes são:

- `THREADS=1`: limita threads internas de bibliotecas numéricas;
- `DOCKER_CPUS=0.75`: limita de verdade o quanto de CPU o container pode usar;
- `DOCKER_MEMORY=2g`: evita que o container consuma memória sem limite;
- `DOCKER_NICE=10`: reduz prioridade do processo dentro do container;
- `PROGRESS_EVERY=25`: mostra progresso a cada 25 imagens.

Comando recomendado:

```
make docker-up
make docker-calibrate
```

Se ainda ficar pesado:

```
make docker-up DOCKER_CPUS=0.50
make docker-calibrate DOCKER_CPUS=0.50
```

Rodar em lotes

Para processar 100 imagens por vez:

```
make docker-up MAX_IMAGES=100 OFFSET=0
make docker-up MAX_IMAGES=100 OFFSET=100
make docker-up MAX_IMAGES=100 OFFSET=200
```

O mesmo vale para calibração:

```
make docker-calibrate MAX_IMAGES=100 OFFSET=0
```

16. Como explicar as escolhas ao professor

Uma forma objetiva de apresentar:

1. O problema não é classificar o frango nem ler texto, é segmentar a região visual do rótulo.
2. Como OCR e ML não são permitidos, a solução usa apenas características visuais clássicas.
3. O pré-processamento melhora contraste, reduz ruído e remove regiões conhecidas de falso positivo.
4. As máscaras procuram evidências diferentes: texto claro, regiões escuras, bordas, brilho e contraste local.
5. Componentes conectados transformam pixels em caixas candidatas.
6. O scoring avalia se uma caixa parece rótulo por cor, textura, borda, contraste, bimodalidade e geometria.
7. O pós-processamento exige uma âncora de produto para evitar brand-only.
8. Depois expande a âncora para incluir o cluster local, porque esse é o alvo final.
9. O crop final sai da imagem original para preservar o dado entregue.
10. O calibrate existe para auditar a decisão visualmente, não para alterar o detector.

17. Limitações conhecidas

Mesmo com esses filtros, o problema é difícil porque:

- há várias embalagens por imagem;
- o plástico cria reflexos;
- alguns rótulos estão parcialmente cobertos;
- há textos parecidos com rótulo, como tabela nutricional ou marca;
- a câmera é fixa, mas a posição das embalagens varia.

Por isso a pipeline prioriza:

- exigir produto dentro do crop;
- evitar brand-only;
- reduzir falsos positivos;
- emitir um cluster primário por frame.

18. Resumo em uma frase

A pipeline usa PDI clássica para encontrar regiões com aparência de rótulo, valida uma âncora obrigatória do produto e expande essa âncora para o cluster visual completo, salvando o crop final a partir da imagem original.